

Atividade 05 — Delivery Tracker: Arquitetura em Camadas

Disciplina: Programação Web II — Backend (Node/Express) · IFAL/Maceió **Capítulo:** 4 — Arquitetura de Software (MVC, Services e Repository Pattern) **Modalidade:** Individual · **Peso:** parte do Exercício Cap. 4 (avaliado) **Correção:** automática (autograder de conformidade) + arquitetura

Contexto

Você foi contratado(a) como pessoa desenvolvedora backend por uma empresa de logística. A empresa precisa **rastrear o ciclo de vida das encomendas** com consistência. Sua tarefa é iniciar a **Delivery Tracker API** — o projeto que evoluirá a cada capítulo do semestre — já com uma **arquitetura em camadas** limpa.

Nesta atividade o foco é a entidade **Entrega** e a separação de responsabilidades: **rotas** → **controllers** → **services** → **repositories**.

Objetivos de aprendizagem

- Estruturar uma API em **camadas** (Controller, Service, Repository).
- Manter a **lógica de negócio no Service** e o **acesso a dados no Repository**.
- Centralizar a **composição das dependências** (injeção de dependência) num único ponto.

Convenções da API (valem para todo o projeto do semestre)

- **Base:** todas as rotas sob `/api`. **Formato:** JSON.
- **Health check:** `GET /api/health` → `200 { "status": "ok" }`.
- **Erros:** sempre `{ "erro": "mensagem" }`.
- **Códigos de status:**

Código	Quando
200	sucesso em GET/PATCH que retorna recurso
201	POST que cria recurso
400	entrada inválida (campo faltando, <code>origem == destino</code>)
404	recurso não encontrado
409	conflito de unicidade (duplicata ativa)
422	violação de regra de estado/negócio (transição inválida)

Contrato de execução (obrigatório — é o que o autograder assume)

1. Instalar com `npm install`.
2. **Subir com** `npm start` (script no `package.json`).
3. **Respeitar** `process.env.PORT` (`process.env.PORT || 3000`).
4. Expor a API sob `/api` e o `GET /api/health`.
5. Branch padrão `main`.

Repositório (comece pelo modelo)

Crie o seu a partir do **repositório-modelo** da atividade: github.com/engsoft-ifal/pweb2-delivery-cap04 → botão **"Use this template"** → nomeie `pweb2-delivery-<matricula>` (ex.: `pweb2-delivery-20231012345`). Ele já vem com o esqueleto `src/`, o **autograder** (`autograder/check.mjs`) e o **workflow do GitHub Actions**. É o mesmo repositório o semestre inteiro (evolui a cada capítulo).

Modelo de domínio

Entrega

Campo	Tipo	Observação
<code>id</code>	number	gerado
<code>descricao</code>	string	obrigatório
<code>origem</code>	string	obrigatório; $\neq$ <code>destino</code>
<code>destino</code>	string	obrigatório
<code>status</code>	enum	<code>CRIADA</code> → <code>EM_TRANSITO</code> → <code>ENTREGUE</code> ; ou <code>CANCELADA</code>
<code>motoristaId</code>	number null	<code>null</code> ao criar (usado na Atividade 06)
<code>historico</code>	Evento[]	<code>{ data: ISO string, descricao: string }</code>

Rotas obrigatórias

Método	Rota	Corpo	Sucesso	Erros
POST	<code>/api/entregas</code>	<code>{ descricao, origem, destino }</code>	201 entrega (status <code>CRIADA</code> , 1 evento no histórico)	400 campos/origem=destino · 409 duplicata ativa
GET	<code>/api/entregas</code>	—	200 array	—
GET	<code>/api/entregas?status=EM_TRANSITO</code>	—	200 array filtrado	—
GET	<code>/api/entregas/:id</code>	—	200 entrega	404
PATCH	<code>/api/entregas/:id/avancar</code>	—	200 entrega (novo status)	404 · 422 transição inválida
PATCH	<code>/api/entregas/:id/cancelar</code>	—	200 entrega (<code>CANCELADA</code>)	404 · 422 já <code>ENTREGUE/CANCELADA</code>
GET	<code>/api/entregas/:id/historico</code>	—	200 array de eventos	404

Regras de negócio

- **Criação:** `origem` $\neq$ `destino` (senão 400); status inicial `CRIADA` ; registra evento no histórico.
- **Duplicidade:** proibida entrega **ativa** (não `ENTREGUE` / `CANCELADA`) com mesma `descricao` + `origem` + `destino` → 409 .
- **Transições:** apenas `CRIADA` → `EM_TRANSITO` → `ENTREGUE` ; qualquer outro avanço → 422 .
- **Cancelamento:** permitido só se `status` $\neq$ `ENTREGUE` e $\neq$ `CANCELADA` (senão 422).

Arquitetura obrigatória

```
src/
├── controllers/    # traduz HTTP ↔ service (sem regra de negócio)
├── services/       # TODA a regra de negócio
├── repositories/   # só acesso a dados
├── database/       # persistência SIMULADA em memória (sem banco real, sem ORM)
├── routes/        # composição das dependências (injeção)
└── utils/
```

- **Regra de negócio só no Service.** Controller não valida regra; Repository não decide regra.
- **Injeção de dependência** no *composition root* (arquivo de rotas): `js const database = new Database(); const repository = new EntregasRepository(database); const service = new EntregasService(repository); const controller = new EntregasController(service);`
- Nesta atividade: **sem banco real, sem ORM** (persistência em memória).

Correção automática (autograder)

A atividade é corrigida por um **autograder de conformidade** que testa a API em **caixa-preta** (bate nos endpoints e confere status e respostas). Ele roda contra uma `BASE_URL`:

```
npm install
npm start # sobe em http://localhost:3000
# em outro terminal:
BASE_URL=http://localhost:3000 node autograder/check.mjs
```

- O autograder (`autograder/check.mjs`) e o workflow do **GitHub Actions** já vêm no repositório-modelo (engsoft-ifal/pweb2-delivery-cap04). A cada `push`, a correção roda sozinha e mostra a nota na aba **Actions**.
- **Meta desta atividade:** todas as checagens de **Entregas** verdes. (As de **Motoristas** ficam para a Atividade 06.)

Entregável

- Repositório `pweb2-delivery-<matricula>` (branch `main`) com o código em camadas.
- `README.md` com instruções de execução e exemplos de requisição (`curl`).

Critérios (transparentes)

Critério	Peso
Autograder — conformidade das rotas de Entregas	60%
Arquitetura em camadas (regra só no Service; Repository só dados)	25%
Injeção de dependência no ponto único (composition root)	10%
README com execução e exemplos	5%